

Práctica 1 EDA

Simón Foguel Giraldo, Pascual Rincón
Cardona

EAFIT

Estructura de Datos y Algoritmos

2026 - 2

Introducción

El presente trabajo constituye un acercamiento a la implementación de estructuras de datos, distinguir entre las complejidades de las operaciones que ofrecen y en qué escenarios resulta conveniente usar una estructura frente a otra considerando sus distintas implementaciones. Se dará solución a dos problemas reales mediante nuestras implementaciones, analizando la complejidad y casos límite, y comparando el rendimiento real con el rendimiento teórico. Para el estudio a realizar, se implementarán las estructuras de datos y se usarán en un programa para procesar un archivo que contiene los casos de prueba generados de forma determinista; se tomarán los tiempos de ejecución y se elaborarán gráficas para determinar el comportamiento del programa. A continuación, se explicarán los problemas a resolver, se presentará la fundamentación teórica, las implementaciones junto al pseudocódigo de las soluciones, análisis de complejidad, experimentación y conclusiones.

Contexto de los problemas:

El problema 1 consiste en una de las principales funciones de un editor de texto, deshacer y rehacer. Es necesario el uso de estructuras de datos, puesto que las operaciones realizadas se deben almacenar en una estructura para poder acceder a estas posteriormente; pero la estructura no puede permitir un acceso arbitrario a sus elementos, se debe tener una norma de acceso a los elementos, dado que la última acción deshecha es la primera que se debe rehacer.

El problema 2 consiste en la implementación de un firewall, un sistema que regula el tráfico de entrada y salida según criterios de ocupación y paquetes recibidos en una ventana de tiempo. Los paquetes se procesan en orden y según el tiempo de procesamiento, el cual es proporcional a su tamaño. La estructura usada debe guardar y eliminar los elementos según su orden de llegada, donde lo primero que llega es lo primero que se procesa. Este tipo de comportamiento es exactamente igual al de una cola.

Fundamentación teórica:

Como se mencionó anteriormente, ambos problemas requieren cierto tipo de estructura con determinadas políticas de acceso. Las estructuras de datos adecuadas para estos problemas son los TDA Stack y Queue; Stack ofrece push (apilar), pop (desapilar) y top (cima); mientras que el Queue ofrece encolar (enqueue), desencolar (dequeue), frente (frente) y back (final) (Weiss, 2014). Las operaciones comunes a ambas estructuras son vacío (empty) y tamaño (size). El Stack resuelve el primer problema, en tanto que Queue resuelve el segundo, dado que sus operaciones cumplen con la política LIFO y FIFO respectivamente.

Stack y Queue son adaptadores de contenedores (Weiss, 2014), es decir, tienen un contenedor subyacente del cual se exponen únicamente las operaciones que se comportan igual que las operaciones de Stack y Queue. El Stack requiere un contenedor que permita inserción y eliminación en un mismo extremo; el Queue, por su parte, debe permitir inserción por un extremo y eliminación por el otro. La elección de la estructura subyacente determina los costos de las operaciones y dependiendo del contexto puede resultar mejor un contenedor u otro. En el estándar de c++ el contenedor por defecto es Deque (cola de doble extremo) (cppreference.com, s.f.-a, s.f.-b), pero este está por fuera del alcance de este curso, por lo cual no se considerará para las implementaciones.

Los contenedores subyacentes que sirven para implementar el Stack son el arreglo dinámico y cualquier tipo de lista enlazada. En cuanto a memoria, el arreglo dinámico se implementa con base en una política de redimensionamiento, donde una vez lleno, todos los datos previos se transfieren a un nuevo bloque de memoria de un tamaño superior. La lista enlazada usa la cantidad justa de espacio, pero cada nodo debe guardar el dato y la dirección del nodo siguiente (y anterior cuando es doblemente enlazada). Esto implica un

overhead considerable, por lo cual, generalmente es preferible el arreglo dinámico, a pesar de que se suele tener espacio sin ocupar. La lista enlazada supera al arreglo cuando el espacio que ocupa el puntero es insignificante con respecto al tamaño del dato a guardar, el cual no es nuestro caso. Con respecto a las operaciones que nos conciernen sobre ambos contenedores, la inserción es de $O(1)$ en ambos casos, aunque cuando se llena el arreglo, es de $O(n)$. Eliminación y ver el elemento en la cima toman $O(1)$.

Para el Queue, el ideal sería cualquier tipo de lista enlazada, dado que usar un arreglo dinámico, elevaría el costo de inserción u eliminación (dependiendo del extremo que se elija como frente) hasta $O(n)$, debido al desplazamiento de todos los elementos almacenados; por lo cual, la mejor alternativa es una lista enlazada. Con respecto a costos, la más eficiente en memoria es la simple, dado que solo requiere un puntero por cada nodo. Es importante señalar que para implementar adecuadamente un Queue sobre lista simple, se debe modificar el estándar y añadir un puntero a la cola de la fila para permitir inserción por un extremo y eliminación por el otro. Con respecto a costos, tanto simple como doblemente enlazada tienen los mismos costos por operación, siendo de $O(1)$.

Ahora, como el problema exige un contenedor de tamaño fijo, se podría añadir un límite al Queue sobre lista enlazada. Sin embargo, la mejor opción es un arreglo estático, sobre el cual se va a iterar de manera circular (más adelante en la experimentación, haremos una comparación de costos). Los índices sobre los cuales se realiza enqueue y dequeue serán atributos de la lista, y cuando uno de estos se salga del máximo, se lo llevará al principio mediante el operador módulo. Esta implementación de Queue no es la convencional y restringe la cantidad de elementos a un número fijo, pero resulta mucho más eficiente, dado que el overhead es mucho menor frente a la lista enlazada y los costos por operación también disminuyen dado que no hace falta actualizar los punteros cada vez que se inserta o elimina un elemento. Más adelante en la experimentación, haremos una comparación de costos.

Habiendo identificado las estructuras necesarias y las implementaciones más eficientes, se procederá con las decisiones de diseño y soluciones a los problemas.

Diseño de las soluciones:

Se implementó la estructura de datos Vector, un arreglo dinámico que duplica su capacidad al llenarse, y la estructura List, la cual es una lista simplemente enlazada que cuenta con un puntero que guarda la dirección del último nodo, permitiendo su uso como contenedor para Stack y Queue. Por otro lado, se implementaron dos Queues, uno sobre List, y un Queue circular. Para el Stack, se implementó con List y con Vector como contenedores subyacentes. Todas las clases fueron implementadas con base en el estándar de `c++`, siguiendo la regla de los cinco (`cppreference.com`, s.f.-c) y exponiendo iteradores para modificación y de lectura (`iterator` y `const_iterator` respectivamente). A pesar de que en las soluciones a ambos problemas no se vayan a utilizar los constructores y operadores de asignación por copia y movimiento, en otros escenarios es indispensable implementarlos cuando se trabaja con memoria dinámica, pues los constructores y operadores por defecto no manejan adecuadamente los punteros, pudiendo resultar en fugas y corrupción de memoria.

Todas las estructuras implementadas tienen sus respectivos reportes de errores para evitar fallos no reportados en el transcurso del programa. Entre los reportes encontramos operaciones de eliminación o visualización sobre contenedor vacío y operaciones de inserción sobre Queue circular lleno.

Para la implementación de las soluciones, se utilizó IA únicamente para corregir errores de compilación y optimizar la entrada y salida de información, para garantizar que los tiempos calculados reflejen el tiempo de procesamiento real y no procesos ajenos a la solución. Con base en las estructuras mencionadas, se implementaron ambas soluciones:

Problema 1:

Para la solución se usaron dos colas, una en la cual se guardan las operaciones Redo y otra para Undo. La dinámica entre ambas colas consiste en que toda acción realizada se añade a Undo, cuando se deshace una acción se guarda en Redo y cuando se realice una nueva acción se limpia el contenido de Redo. Cuando se almacena el dato de un cambio en una de las pilas, se guarda el fragmento modificado y no la totalidad del documento para reducir costos. Las operaciones insert, delete y replace fueron reducidas a la función modificar, que mediante sus parámetros permite ejectutar cualquiera de los tres eventos. Las ediciones nulas no tienen ningún efecto en el programa además de ser registradas por el log. Siempre se evalúa que se pueda realizar una operación para evitar que se rompa el programa debido una excepción.

La solución está compuesta por varios módulos los cuales se encargan de la ejecución del programa, lectura de datos, creación de eventos y reporte final. Desde los ejecutables se llama el motor, el cual se encarga de ejecutar los componentes, y desde el ejecutable se calcula la información de interés.

Problema 2:

Para este problema, dado que el enunciado no establecía que evento ocasionaba el procesamiento de un archivo, se realizaron varias suposiciones. En primer lugar, el buffer tiene una tasa de procesamiento o velocidad de red R , la cual se usa para asignar un tiempo de salida a cada archivo según su tamaño y en función del tiempo de llegada y del tiempo de salida del paquete previamente agregado al buffer. Segundo, se asumió que el parámetro C representa la cantidad de paquetes que puede almacenar el buffer, no la capacidad de bytes de este. Asimismo, las marcas de tiempo son estrictamente no decrecientes y para determinar si un paquete ya se puede sacar del buffer, se compara el frente de la cola con el tiempo de llegada del paquete más reciente, y mientras que este último sea mayor, se hace dequeue. Posteriormente, se revisa que el buffer no esté lleno, se purgan las marcas de tiempo menores a $t - T$ y se evalúa que el tamaño de la lista resultante sea menor que L ; de no ser así, se rechaza por exceso de tasa. Si se pasan ambos criterios, se acepta el paquete y se registra la marca de tiempo. Se tomaron estas decisiones dado que permiten simular de manera simple el comportamiento de un buffer, a partir de políticas de tasa y ocupación únicamente. Además, mediante la variación de los parámetros, podemos cambiar el comportamiento del buffer para generar los casos de prueba exigidos sin cambiar necesariamente los datos de entrada. Para evitar demoras por lectura de información, se carga toda la información a un Vector, previniendo operaciones costosas ocasionadas por el operador de extracción.

Para las pruebas no se ejecutaron los casos que exigían realizar dequeue sobre un buffer vacío o push sobre buffer lleno, pues por implementación, ambas operaciones arrojarían una excepción interrumpiendo la ejecución del programa. Asimismo, por decisión de diseño no existe evento que pueda ocasionar dequeue sobre buffer vacío y antes de hacer enqueue se revisa que no esté lleno el buffer. Adicionalmente, al decidir eliminar las marcas menores a $t - T$, si un paquete llega en el borde, la marca correspondiente a t_0 no se eliminaría y la marca de tiempo se encolaría en caso de que la cantidad de marcas en la ventana sea menor o igual a L .

Análisis de complejidad:

A continuación, se listarán las operaciones involucradas en la resolución de ambos problemas y sus complejidades para mejor, peor y caso promedio.

Problema 1:

- Push sobre arreglo sin redimensionamiento

Para realizar push se debe insertar un dato en el índice correspondiente al tamaño actual y se debe aumentar el tamaño en 1. La complejidad de dicha operación es de $\Theta(1)$. La complejidad espacial es de $\Theta(1)$, puesto que el tamaño del bloque de memoria es invariante.

- Push sobre Vector

Siempre que haya espacio disponible, la inserción es una operación extremadamente eficiente, pero cuando no queda espacio, se deben desplazar todos los elementos a un nuevo bloque de memoria de mayor tamaño. Al

definir la política de redimensionamiento como duplicación de la capacidad, es cada vez menos frecuente que haya que realizar este redimensionamiento, resultando en una operación generalmente muy eficiente con eventos costosos cada cierto tiempo.

Al realizar el desplazamiento de los elementos a un nuevo bloque del doble de tamaño, la cantidad de operaciones a realizar es igual a la capacidad del bloque anterior. Asumiendo que n representa la cantidad de inserciones realizadas y que es una potencia de 2, tenemos que el costo de los eventos de redimensionamiento, que consiste en la copia de todos los elementos previamente almacenados, es de

$$1 + 2 + 4 + 8 + \dots + \frac{n}{2} = n - 1$$

Ahora, el costo de las operaciones de inserción simple es de n . Por lo cual, el costo total de n inserciones viene dado por la suma de ambos costos, obteniendo en total

$$n + (n - 1) = 2n - 1 = O(n)$$

Al dividir este resultado entre la cantidad de datos obtenemos

$$\frac{O(n)}{n} = O(1)$$

Por lo tanto, la complejidad amortizada para esta operación es de $O(1)$.

En cuanto a la complejidad espacial, la mayoría de los casos es de $O(1)$ dado que suele haber espacio disponible para la inserción. Cuando es necesario redimensionar, la complejidad espacial es de $O(n)$, dado que es el tamaño extra que se debe reservar.

- **Push sobre Lista enlazada**

Realizar push sobre lista enlazada consiste en crear un nuevo nodo, actualizar el atributo next del nodo cabeza, y sumarle 1 al tamaño. Considerando que el número de operaciones a realizar siempre es el mismo, la complejidad viene dada por $\Theta(1)$. La complejidad espacial es independiente del número de datos dado que solo se crea un nodo por inserción, por lo cual la complejidad espacial también corresponde a $\Theta(1)$.

- **Pop y peek sobre vector**

Pop es equivalente a usar `pop_back` sobre el atributo vector, que, por implementación, consiste en restarle 1 al tamaño del Vector y peek simplemente es retornar el elemento al final del vector. Ambas operaciones se realizan en una sola instrucción, por lo cual su tiempo de ejecución es de $\Theta(1)$. Igualmente, el espacio se mantiene constante, por lo cual la complejidad espacial es de $\Theta(1)$.

- **Pop y peek sobre lista enlazada**

Peek requiere retornar el elemento al frente, lo cual toma una operación. Pop requiere guardar en una nueva variable el nodo al frente de la lista, reasignar punteros (debido al nodo centinela), eliminar el nodo guardado y restar 1 al tamaño. Ambas operaciones toman una cantidad constante de pasos, por lo cual su complejidad es de $\Theta(1)$. En cuanto a memoria, ambas toman $\Theta(1)$, dado que no se requiere memoria adicional.

- **Empty**

Empty, sobre ambas implementaciones, solo requiere una comparación, por lo cual toma $\Theta(1)$. No requiere espacio extra, por lo cual su complejidad espacial es de $\Theta(1)$.

- **Complejidad global**

La complejidad global del problema depende enteramente de la naturaleza de los flujos de entrada. En este caso vamos a considerar dentro de la complejidad únicamente el tiempo tomado por las pilas de redo y undo dado que las operaciones asociadas a la modificación del documento no tienen que ver con el procesamiento de las operaciones. Con base en lo anterior, la complejidad del programa sería de $O(n)$, dado que el programa se basa

en realizar push y pop sobre ambas pilas de la implementación, operaciones las cuales toman $O(1)$. Los flujos de entrada fueron programados de tal manera que se evitarán operaciones fallidas, lo cual implica que siempre se realizara push o pop. En los casos que sea necesario realizar clear sobre la pila de redo, la complejidad de esta operación a lo largo del programa viene dada por $O(n)$, dado que cada operación de edit entra y sale una única vez. La complejidad que se obtendrá en el apartado de experimentación toma en cuenta las modificaciones sobre el string, por lo cual el tiempo obtenido experimentalmente dará cuenta del procesamiento más tiempo de escritura. En el peor caso, o sea inserciones o eliminaciones al comienzo del documento, resultan en una complejidad de $O(n^2)$, debido a que inserciones al comienzo toman $O(n)$ y se ejecutan n veces. En el mejor caso la complejidad es de $O(n)$, que corresponde a inserciones y eliminaciones al final del string.

La complejidad espacial es proporcional al número de datos, puesto que cada operación se debe almacenar. Es por esto que la complejidad espacial viene dada por $O(n)$.

Sabiendo esto, se esperaría que, para cada instancia solicitada, el tiempo de ejecución aumentará por un factor entre 10 y 100. También, observaremos que, a pesar de que la complejidad se mantenga igual, para la implementación basada en lista enlazada el tiempo de ejecución será mayor debido a un mayor número de instrucciones por operación.

Problema 2:

- Enqueue (asumiendo que no está llena) y dequeue sobre cola circular.

Para mejor, peor y caso promedio, tenemos $\Theta(1)$, dado que los tres casos son exactamente iguales y la cantidad de operaciones es constante: insertar y mover el índice para enqueue, y únicamente mover el índice para dequeue. La complejidad espacial de ambas operaciones es $\Theta(1)$, dado que no necesitan memoria nueva por el bloque de memoria estático.

- Enqueue y dequeue sobre Queue con lista enlazada simple.

Para los tres casos, nuevamente tenemos $\Theta(1)$, dado que la cantidad de operaciones a realizar siempre es la misma. A pesar de esto, realizar estas operaciones sobre esta implementación de Queue implica una mayor cantidad de pasos debido al movimiento de punteros y liberación de memoria. Para inserción y eliminación las operaciones son las siguientes: Crear nuevo nodo o eliminar nodo, actualizar cabeza o cola, y aumentar o disminuir el tamaño. Como vemos, una operación sobre lista enlazada simple acarrea muchas más instrucciones, por lo cual su rendimiento es menor. La complejidad espacial es de $\Theta(1)$, dado que inserción requiere reservar únicamente un nuevo nodo, y eliminación guardar el nodo a eliminar temporalmente.

- Full y Empty

Para ambas implementaciones $\Theta(1)$, dado que ambas se pueden realizar mediante una sola comparación entre los atributos de las clases.

- Purgar expirados

Para determinar de forma verosímil el costo de esta operación, se debe utilizar el análisis amortizado, dado que de otra manera obtendríamos una respuesta engañosa, pues en el peor de los casos esta operación toma $O(n)$, y al procesar n datos tomaría $O(n^2)$, lo cual es altamente ineficiente y no da cuenta de la complejidad de la operación a lo largo del programa.

Cada marca de tiempo entra y sale al menos una vez del buffer. En otras palabras, esta operación ejecutará a lo sumo $O(n)$ operaciones (considerando comparaciones y asignaciones, en total 3). Por lo cual, tomando el número de datos, obtenemos que por dato la complejidad es de $\frac{O(n)}{n}$, resultando en una complejidad de $O(1)$ amortizada.

- Procesamiento paquetes

Para nuestra implementación, es necesario analizar una operación adicional, el procesamiento de paquetes. Dado que la cantidad de veces que se ejecuta el ciclo de procesamiento depende de la naturaleza de los datos, su comportamiento varía para entradas diferentes. Además, el rendimiento en el peor caso no da cuenta del rendimiento de esta operación durante la ejecución del programa. Por esto es necesario usar análisis amortizado.

Nuevamente, cada dato entra y sale a lo sumo una vez del buffer, es decir, se ejecutarán a lo sumo $O(n)$ operaciones (considerando las comparaciones realizadas por iteración). Al dividir este resultado entre el número total de datos, obtenemos una complejidad de $O(1)$.

- Complejidad global

La complejidad global es de $\Theta(n)$, dado que la cota superior es de $O(n)$ como demostramos mediante el análisis amortizado, y la cota inferior es de $\Omega(n)$ dado que el caso trivial corresponde al rechazo de todo archivo, para lo cual, el programa ha de evaluar los criterios definidos por cada paquete recibido un total de n veces. Dado que la misma función sirve de cota superior e inferior, el resultado es de $\Theta(n)$.

Si se implementa la cola de paquetes sobre lista enlazada se tendría también, $\Theta(n)$ pero con un mayor número de operaciones, aumentado el tiempo de ejecución. Al depender la ocupación de ambas colas de los parámetros, la complejidad espacial depende directamente de los parámetros, y a pesar de que el tamaño vaya aumentando a medida que entran datos, llega un punto en el que deja de aumentar la ocupación, manteniéndose constante. Para el peor caso tenemos ocupación total, pero con determinado límite establecido por los parámetros, y para el mejor tenemos el caso en el que no se recibe nada, siendo ambos números fijos. Con base en esto, la complejidad espacial es de $\Theta(1)$.

Experimentación:

Problema 1:

Para el desarrollo experimental se ejecutará el programa con los tamaños de instancia sugeridos al menos 5 veces alternando entre pila sobre cola y pila sobre arreglo, reportando los tiempos solicitados, no-ops, redos y undos efectivos. Posteriormente se ejecutarán los casos de prueba sugeridos, reportando log de eventos y estado final del documento. Los flujos de entrada fueron calibrados de tal manera que la edición sea preponderante frente a cualquier otro tipo de operación, garantizando que la cantidad de operaciones efectivas sea apreciable, y se generaron manualmente los casos de prueba solicitados añadiendo sus respectivos reportes.

N	Pila	Media ms	Desv ms	Efectivos	No-ops
1.000	Arreglo	1.650	0.690	178	79
1.000	Lista	1.205	0.168	178	79
10.000	Arreglo	11.202	0.985	1643	823
10.000	Lista	9.290	0.967	1643	823
100.000	Arreglo	241.469	3.824	16422	8545
100.000	Lista	228.151	4.289	16422	8545
1.000.000	Arreglo	16157.975	844.547	164875	84945
1.000.000	Lista	16307.203	1387.592	164875	84945

Caso	Estado final documento	Log operaciones	Resumen
1	chau	UNDO aplicado Pila vacia, REDO sin efecto	elementos en pila Undo: 3 elementos en pila Redo: 0 UNDO/REDO efectivos: 1 UNDO/REDO no-op: 1
2		Pila vacia, UNDO sin efecto Pila vacia, UNDO sin efecto Pila vacia, UNDO sin efecto	elementos en pila Undo: 0 elementos en pila Redo: 0 UNDO/REDO efectivos: 0 UNDO/REDO no-op: 3

3	hola	UNDO aplicado REDO aplicado	elementos en pila Undo: 1 elementos en pila Redo: 0 UNDO/REDO efectivos: 2 UNDO/REDO no-op: 0
4	primerotercero	UNDO aplicado Pila vacia, REDO sin efecto	elementos en pila Undo: 2 elementos en pila Redo: 0 UNDO/REDO efectivos: 1 UNDO/REDO no-op: 1
5		UNDO aplicado UNDO aplicado UNDO aplicado UNDO aplicado UNDO aplicado	elementos en pila Undo: 0 elementos en pila Redo: 5 UNDO/REDO efectivos: 5 UNDO/REDO no-op: 0
6	uno	UNDO aplicado REDO aplicado Pila vacia, REDO sin efecto Pila vacia, REDO sin efecto Pila vacia, REDO sin efecto	elementos en pila Undo: 1 elementos en pila Redo: 0 UNDO/REDO efectivos: 2 UNDO/REDO no-op: 3
7		20 undos	elementos en pila Undo: 0 elementos en pila Redo: 20 UNDO/REDO efectivos: 20 UNDO/REDO no-op: 0

Análisis resultados:

Podemos corroborar que el tiempo de ejecución es efectivamente el mencionado, dado que entre cada instancia el tiempo de ejecución aumenta entre 10 y 100 veces. Contrario a lo esperado, el tiempo de ejecución fue menor para la lista enlazada que para el arreglo. Podemos atribuir esto al hecho de que se están cronometrando las operaciones sobre el string lo cual no da cuenta de las operaciones sobre las pilas de manera aislada. Otra posibilidad es el hecho de que el costo del puntero a un nodo puede ser considerablemente menor que los elementos a almacenar dentro de las clases Evento y Registro volviendo más costosas las operaciones de redimensionamiento. Finalmente, podemos observar los resultados obtenidos para los casos de prueba realizando las modificaciones sobre un string, y realizando catch sobre las excepciones para evitar que el programa se rompa

Problema 2:

Para el desarrollo experimental se ejecutará el programa con los tamaños de instancia sugeridos. Se obtendrán los tiempos de ejecución al menos cinco veces para cada prueba y se calculará el promedio y la desviación estándar, y se contará el número de paquetes aceptados y rechazados según su causa. Para el caso $n = 1.000.000$, se generarán gráficas para observar los flujos de entrada al programa y observar cómo se procesan. Es importante señalar que las marcas de tiempo reportadas serán las generadas sintéticamente, pues así podemos estudiar el comportamiento real de un firewall y el comportamiento del flujo de entrada.

Los flujos sintéticos emulan el comportamiento de los flujos reales, presentando saturaciones y momentos de calma donde el flujo de entrada está por debajo de la tasa máxima. Se ejecutará el programa para cada instancia con los siguientes parámetros: $C = 200$, $T = 1000$, $L = 150$, $R = 500$, pues permiten simular el comportamiento real de un buffer con base en las entradas generadas. Únicamente para el caso de prueba 1.000.000 variaremos los parámetros del programa.

Comparación instancias

N	Desviación estándar (ns)	Promedio (ns)	Aceptados	Rechazo buffer	Rechazo tasa	Ocupación máxima
1.000	24621.7302	105620	300	0	700	19
10.000	168820.460	1165800	2634	0	7366	19

100.000	362460.841	8911720	26400	0	73600	19
1.000.000	1988626.79	89928500	262987	0	737013	19

Comparación tiempo Queue circular y Queue normal

N	Queue circular (ns)	Queue normal (ns)
1.000	134200	141600
10.000	1063300	1522300
100.000	8713500	12462200
1.000.000	97596100	114879900

Experimentos n = 1.000.000 (Parámetros: C, T, L, R respectivamente)

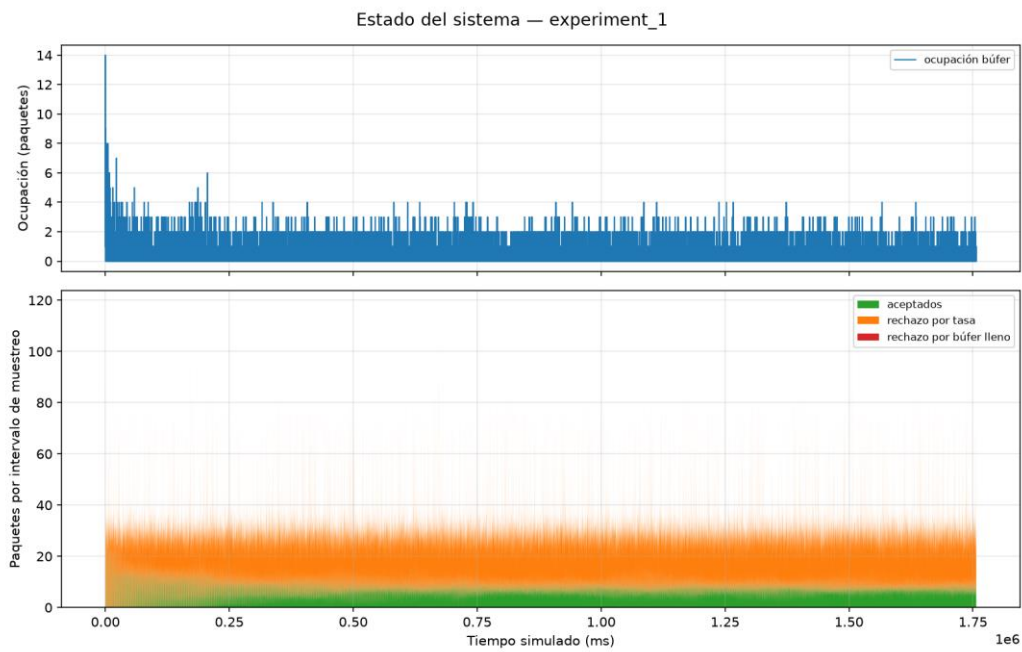
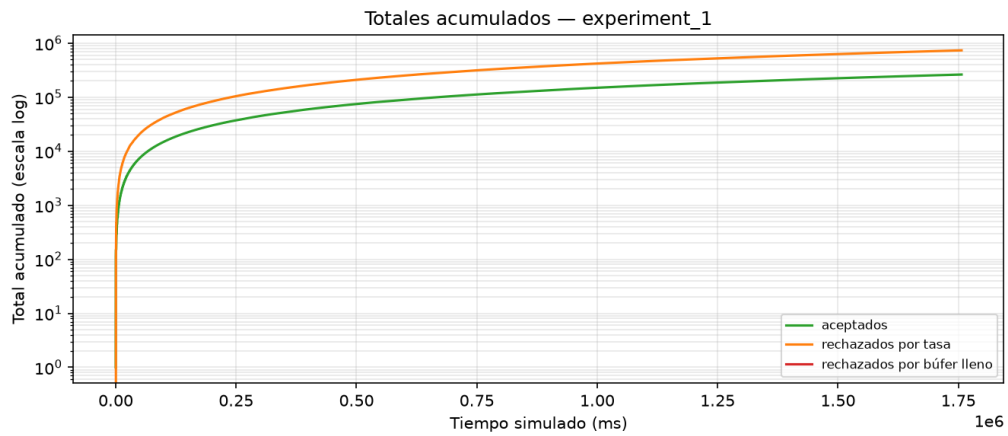
Parámetros	Aceptados	Rechazados buffer	Rechazados tasa	Ocupación Maxima
200, 1000, 150, 500	262987	0	737013	19
150, 800, 200, 100	224461	775310	229	150
250, 600, 150, 150	337003	661813	1184	250
100, 700, 200, 700	499667	0	500333	6

Análisis resultados:

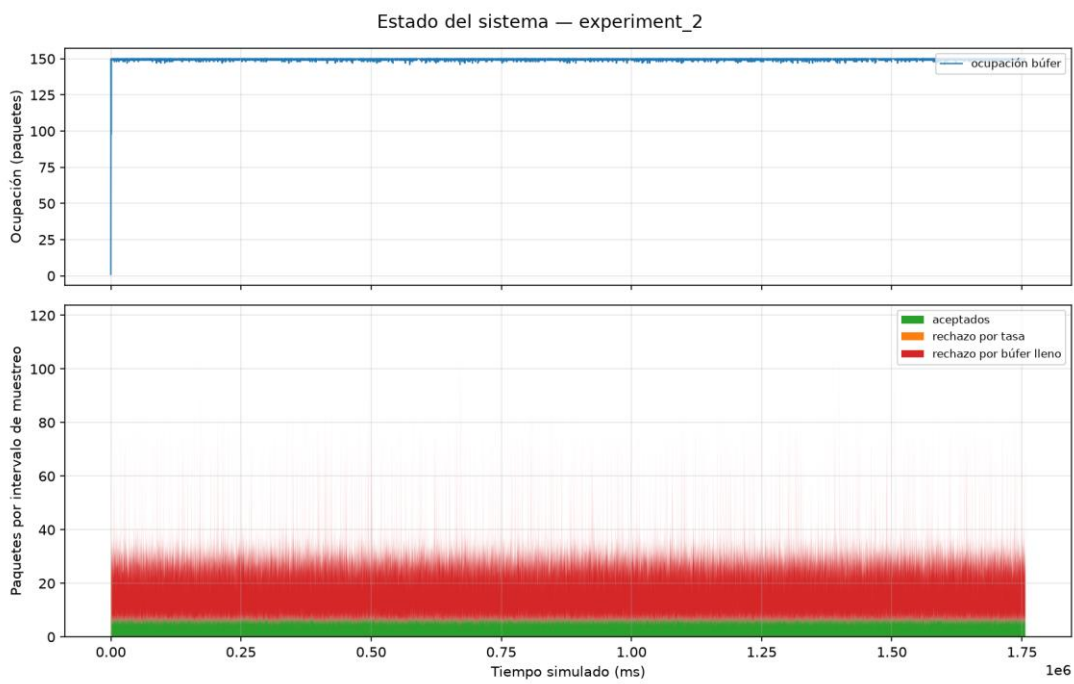
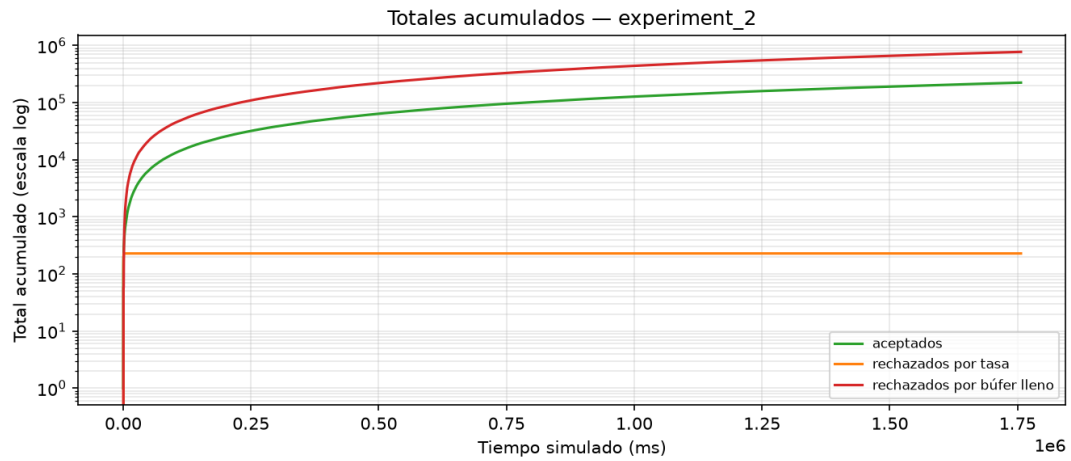
En primer lugar, podemos confirmar que la complejidad determinada analíticamente es correcta; al calcular los cocientes $\frac{T(10n)}{T(n)}$, siendo $T(n)$ el tiempo de ejecución del programa en función del número de datos, obtenemos 11.037, 7.644, 10.09 para n igual a 1.000, 10.000 y 100.000 respectivamente. Al ser la complejidad del programa lineal, se esperaría que un aumento de la cantidad de datos produjera un aumento proporcional en el tiempo de ejecución, el valor esperado para este cociente es de 10. La discrepancia entre los valores obtenidos experimentalmente y el valor teórico se debe a que por cada paquete recibido no se realiza la misma cantidad de operaciones, por lo cual la cantidad total de operaciones no es directamente proporcional al tamaño de datos resultando en la diferencia mencionada. Además, factores como jitter del SO, calendarización de procesos y caché fría afectan la consistencia entre los tiempos de ejecución obtenidos, por lo tanto, en la práctica nunca se va a obtener el mismo resultado encontrado analíticamente. Sabiendo esto, los valores obtenidos respaldan el análisis teórico, pues la variación promedio es de 11,61%, siendo un valor consistente considerando los factores que afectan las mediciones.

En segundo lugar, podemos corroborar que la Cola sobre lista enlazada es mucho menos eficiente en comparación a la Cola circular, debido al mayor número de instrucciones que se deben realizar para encolar y desencolar sobre una lista enlazada.

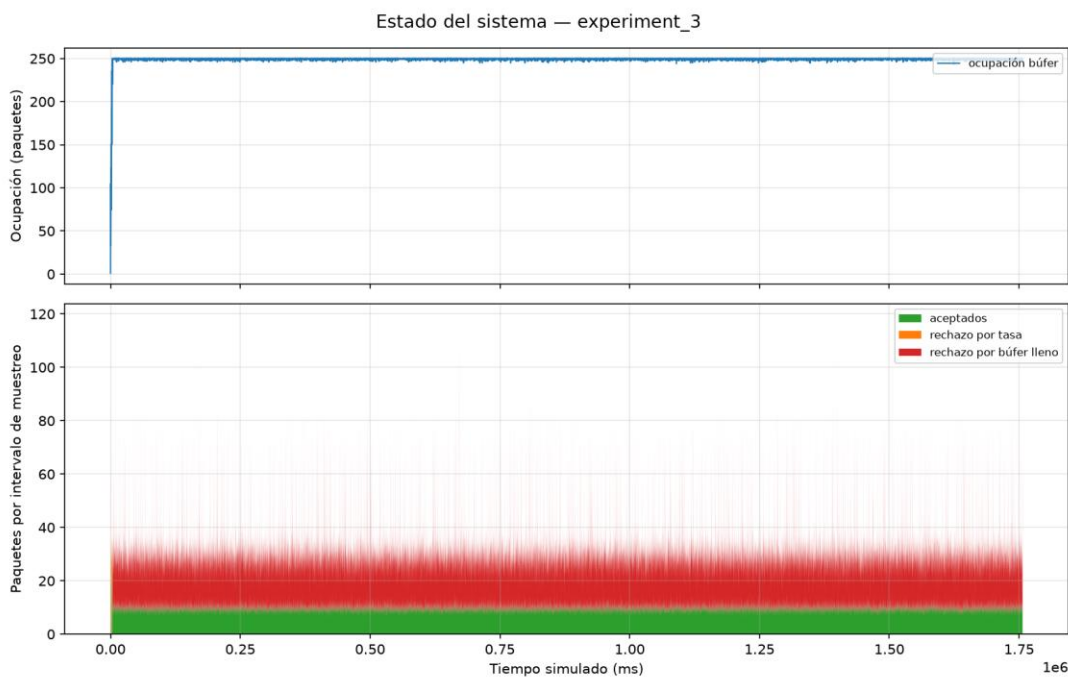
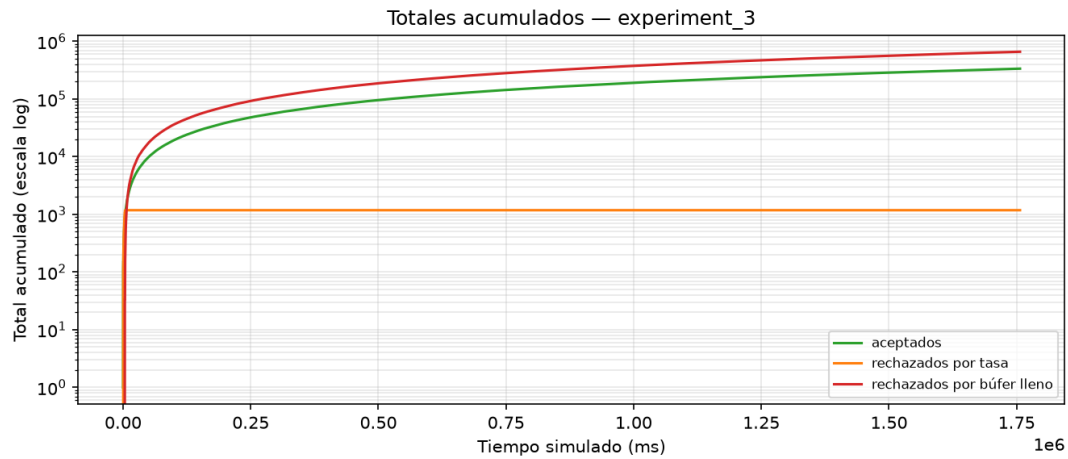
Finalmente, para analizar el comportamiento del programa frente a diferentes flujos de llegada, se variaron los parámetros del buffer, para inducir los casos de prueba solicitados. Para el primer caso, tenemos un flujo por debajo de la capacidad máxima del buffer, pero por encima del exceso de tasa. Como se puede ver en la gráfica 1, los aceptados y rechazados por tasa crecen de forma constante a lo largo del programa, y la máxima ocupación ocurre al principio.



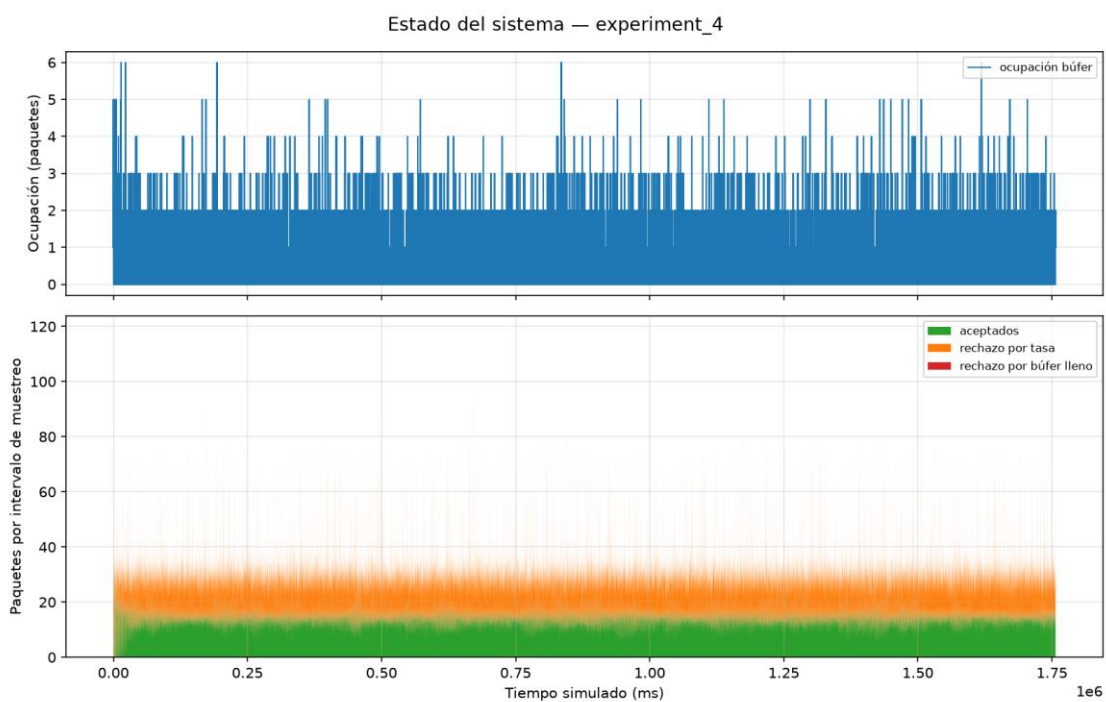
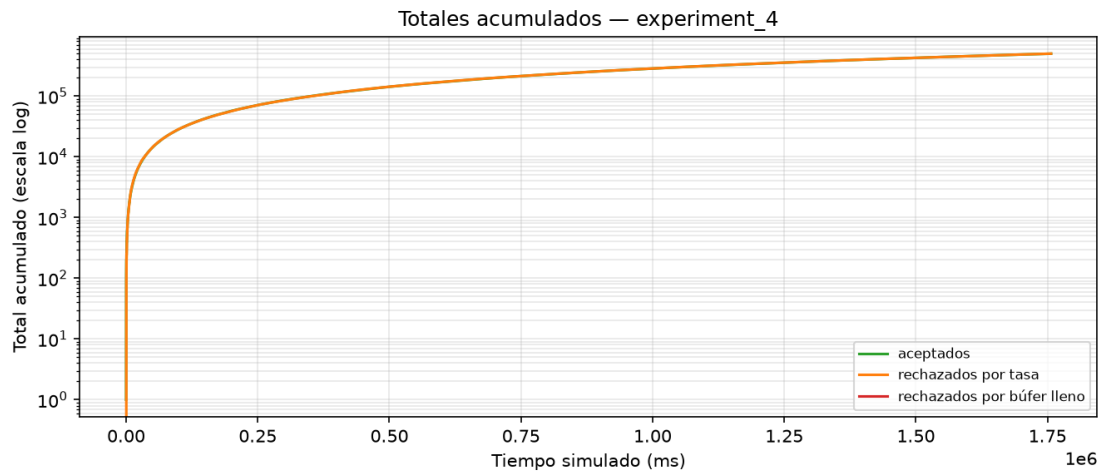
Para el segundo, el buffer procesaba mucho más lento los archivos, resultando en más rechazos por ocupación que por tasa. En la gráfica 2 se observa que, durante la ejecución, los rechazos por tasa se dieron al inicio, antes de que se llenara el buffer. Posteriormente, los paquetes fueron rechazados por ocupación. Esto corresponde a un flujo que excedía la tasa al principio y posteriormente la ocupación.



En la gráfica 3, tenemos un flujo que corresponde a un flujo inicial por encima del exceso de tasa, sucedido por un flujo que llena el buffer debido a una tasa de procesamiento relativamente baja. Por eso, al principio los rechazos fueron por tasa hasta que se llena el buffer y los rechazos reportados son por ocupación.



Finalmente, la gráfica 4 corresponde a un flujo que está por debajo de la ocupación máxima por la tasa de procesamiento alta, y que va alternando entre exceso de tasa y entrada de paquetes. El rechazo por tasa y número de paquetes aceptados crece de forma constante e igual a lo largo de la ejecución del programa.



Conclusiones

En primer lugar, pudimos implementar adecuadamente las estructuras solicitadas con diferentes contenedores subyacentes según el estándar de c++, garantizando buen manejo de memoria. Además, se implementaron los iteradores solicitados para recorrer las estructuras. Evidenciamos los beneficios y desventajas asociadas al uso de bloques de memoria contiguos y nodos dispersos en la memoria, resultando más eficiente para colas y pilas la implementación sobre nodos cuando el tamaño del elemento a almacenar es considerablemente mayor que el tamaño del puntero al siguiente nodo par. Identificamos las mejores alternativas de implementación, encontrando que para mayor flexibilidad y eficiencia era posible añadir un puntero al último nodo, permitiendo el uso de la lista simplemente enlazada para pila y cola manteniendo las operaciones asociadas a ambas estructuras en $O(1)$.

Aprendimos que hay escenarios en los que la complejidad de una operación se debe considerar a lo largo de la ejecución del programa puesto que un análisis de peor caso puede sobreestimar el tiempo de

ejecución y que el análisis teórico no da cuenta de los tiempos de ejecución exactos, pues existen procesos que pueden afectar el rendimiento y los tiempos de ejecución pueden depender de la naturaleza de los datos; por ello la complejidad no permite determinar que algoritmo es más eficiente pero si como se comportará a medida que crecen los datos a procesar. Adicionalmente, reforzamos nuestros conocimientos con respecto al lenguaje y sus diferentes usos como memoria dinámica, implementar estructuras que usan memoria dinámica y funciones de la Standard Library, como `exchange`, `move` y `swap`, utilizadas para ceder recursos en operaciones con Rvalues.

Finalmente, para una implementación de mayor alcance se puede ofrecer mejor manejo de excepciones, incorporar operaciones adicionales como inserción mediante iteradores o la función de `reserve` para vector, y mejorar el uso de la memoria dinámica mediante funciones como `operator new` o `std::malloc`, las cuales reservan el bloque de memoria en crudo, evitando la creación de objetos por defecto en cada espacio de memoria. Seguir esta recomendación elevaría la eficiencia de las implementaciones considerablemente, siendo indispensable en escenarios de mayor escala.

Reporte aportes:

Pascual Rincón Cardona

Estructuras de datos, solución problema 2, redacción documento completo.

Simón Foguel Giraldo

Correcciones estructuras de datos, solución problema 1, pseudocódigo, Readme.

Referencias:

cppreference.com. (s.f.-a). *std::queue*. Recuperado el 31 de agosto de 2026, de <https://en.cppreference.com/w/cpp/container/queue>

cppreference.com. (s.f.-b). *std::stack*. Recuperado el 31 de agosto de 2026, de <https://en.cppreference.com/w/cpp/container/stack>

cppreference.com. (s.f.-c). *Rule of three/five/zero*. Recuperado el 31 de agosto de 2026, de https://en.cppreference.com/w/cpp/language/rule_of_three

Weiss, M. A. (2014). *Data structures and algorithm analysis in C++* (4.^a ed.). Pearson.

Declaración uso inteligencia artificial: El presente trabajo se hizo con ayuda de herramientas de inteligencia artificial para discutir implementaciones, señalar optimizaciones de diseño, creación de flujos de entrada para simular escenarios reales y discusión de complejidad. No se copió nada realizado por IA a excepción del generador de gráficos y generador de flujos sintéticos, y se procuró un entendimiento absoluto de lo sugerido por la IA, lo cual se puede constatar.

Declaración Pascual Rincón Cardona:

GPT-5.6 Luna

<https://chatgpt.com/share/6a95cb27-ca4c-83e9-a3cc-8871db217767>

<https://chatgpt.com/share/6a95ca35-d8fc-83e9-b707-dccc1d21cded>

<https://chatgpt.com/g/g-p-6a7cb967fa3081919e066abceffe265f-trabajo-estructura-de-datos/c/6a7fa596-e3ac-83e9-90ab-330489c31415>

<https://chatgpt.com/share/6a95cafa-d4d4-83e9-aa0f-b60a9c1b536b>

<https://chatgpt.com/g/g-p-6a7cb967fa3081919e066abceffe265f-trabajo-estructura-de-datos/c/6a95ccad-93e4-83e9-afb1-9f09b3fcba67>

Sonnet 5 Medio

<https://claude.ai/share/2da0fb23-a258-4ff3-ae90-015f4e6dbe32>

Declaración Simón Foguel:

Se encuentra en la carpeta report del repositorio.